Unidad 5: Pruebas y Manejo de Errores

Tema 5.2: Programación Defensiva e Informe de Errores

Área de Ciencias de la Computación

Facultad de Ingeniería, UASLP

27 de octubre de 2026

¿Qué es la Programación Defensiva?

Definición

Es un enfoque de diseño que busca garantizar el comportamiento predecible y seguro de un programa, incluso ante situaciones imprevistas, usos incorrectos o entradas de datos inválidas.

Principios Fundamentales:

- **Asumir lo peor:** Todo dato externo (usuario, archivo, red) es sospechoso hasta que se demuestre lo contrario.
- **Validación estricta:** Revisar parámetros y estados antes de ejecutar la lógica de negocio.
- **Fallo seguro:** Si un error es inevitable, el sistema debe detener la operación anómala de forma controlada.

¿Qué son las excepciones en Java?

Definición Fundamental

Las excepciones en Java son eventos anómalos que pueden ocurrir durante la ejecución de un programa y que alteran el flujo normal de ejecución.

Estos eventos representan **situaciones inesperadas o errores** que deben ser manejados de manera adecuada para:

- Garantizar que el programa continúe ejecutándose.
- Evitar interrupciones y caídas del sistema (*crashes*).

Un Ejemplo Cotidiano: Inicio de Sesión

Imagina que estás utilizando una aplicación que requiere que ingreses una contraseña para acceder a tu cuenta.

Escenario 1: El Flujo Esperado

- Ingresas la contraseña correcta.
- La aplicación te permite acceder a tu cuenta y continúa sus funciones de manera normal.
- **Resultado:** Este es el flujo ideal y **no genera una excepción**.

Un Ejemplo Cotidiano: La Excepción

Escenario 2: El Flujo Alterno (Error)

- Ingresas una contraseña incorrecta.

Sin manejo de excepciones

La aplicación se cierra inesperadamente o falla de manera abrupta.

Con manejo de excepciones

Ves un mensaje de error indicando que la contraseña es incorrecta y te da la oportunidad de intentarlo nuevamente. La aplicación ha detectado el evento y lo ha resuelto de forma elegante sin detenerse.

Importancia de las excepciones (1/2)

Las excepciones son elementos fundamentales en la construcción de programas robustos y fiables en cualquier lenguaje de programación, su importancia radica en varios aspectos:

- **Manejo de errores de manera estructurada:** Las excepciones brindan una forma estructurada de manejar errores y situaciones excepcionales. Esto permite separar el código de manejo de errores del código funcional, lo que resulta en un código más limpio y fácil de entender.
- **Mejora de la robustez y confiabilidad:** Al anticipar y manejar situaciones excepcionales de forma adecuada, podemos hacer que nuestras aplicaciones sean más resistentes y confiables. Esto es esencial para garantizar que el programa funcione de manera predecible y sin interrupciones.

Importancia de las excepciones (2/2)

- **Retroalimentación al usuario:** Mediante el uso de excepciones, podemos proporcionar mensajes de error significativos y detallados al usuario, lo que facilita la comprensión de lo que salió mal y cómo corregirlo. Esto mejora la experiencia del usuario y facilita la resolución de problemas.

- **Flexibilidad y adaptabilidad:** El manejo adecuado de excepciones permite a los desarrolladores adaptar el comportamiento de la aplicación según las circunstancias. Esto significa que podemos cambiar la forma en que se manejan las excepciones en diferentes partes del programa para adaptarse a los requisitos específicos de cada sección.

Tipos de excepciones en Java (1/2)

Las excepciones se dividen en tres categorías principales, cada una heredada de la clase base `Throwable`. Estas categorías son:

- **Checked exceptions:** Son excepciones que se deben declarar en la firma del método o capturar explícitamente en un bloque `try-catch`. Si una excepción comprobada no se maneja correctamente, el código no se compilará. Estas excepciones se heredan de la clase `Exception`.

Tipos de excepciones en Java (2/2)

- **Unchecked exceptions:** Son excepciones que no están obligadas a ser manejadas explícitamente. Estas excepciones ocurren durante la ejecución del programa y no se requiere que sean declaradas en la firma del método o capturadas mediante un bloque `try-catch`. Se heredan de la clase `RuntimeException`.

- **Errors:** Son problemas graves que generalmente están fuera del control del programador y no deben manejarse explícitamente. Estos errores indican problemas serios que deberían detener la ejecución del programa. Se heredan de la clase `Error`.

Excepciones Comunes

ArithmeticException	**Excepción de una operción aritmética (ej. División por cero)**
ArrayStoreException	**Excepción en una operación en el uso de un array**
NegativeArraySizeException	**Excepción en una operación en el uso de un array**
NullPointerException	**Excepción producida por una referencia a un objeto nulo**
EOFException	**Excepción producida por alcanzar el final de un fichero**
IOException	**Excepción en un proceso de entrada y salida**
FileNotFoundException	**Excepción por no encontrar un fichero**
NumberFormatException	**Excepción en la conversión de una cadena de caracteres a un valor numérico**

Manejo de excepciones en Java

El manejo de excepciones en Java es fundamental. A continuación, veremos cómo manejar excepciones utilizando bloques `try`, `catch` y `finally`.

El bloque `try` se utiliza para envolver el código que puede generar una excepción. Si se produce una excepción dentro del bloque `try`, se captura en uno o más bloques `catch` asociados.

```
1 try {
2     // Codigo que puede generar una excepcion
3     int resultado = 10 / 0; // Esto generara una excepcion de
      division por cero (ArithmeticException)
4 } catch (ArithmeticException e) {
5     // Captura de la excepcion especifica
6     System.out.println("Ocurrio una excepcion: " + e.getMessage());
7 }
8
```

El bloque finally

Por otro lado, el bloque `finally` se utiliza para ejecutar código que debe ejecutarse, independientemente de si se ha producido una excepción o no. Se usa para liberar recursos que deben cerrarse, como conexiones a bases de datos o archivos.

```
1  try {
2      // Codigo que puede generar una excepcion
3      // ...
4  } catch (Exception e) {
5      // Captura de cualquier excepcion
6      System.out.println("Ocurrio una excepcion: " + e.getMessage());
7  } finally {
8      // Codigo que se ejecutara siempre, incluso si se produjo una
       excepcion
9      // Liberar recursos
10 }
11
```

Múltiples bloques catch

Podemos tener varios bloques `catch` para manejar diferentes tipos de excepciones de manera específica.

```
1  try {
2      // Codigo que puede generar una excepcion
3      // ...
4  } catch (ArithmeticException e) {
5      // Captura de excepcion de division por cero
6      System.out.println("Ocurrio una excepcion de division por cero:
        " + e.getMessage());
7  } catch (Exception e) {
8      // Captura de cualquier otra excepcion
9      System.out.println("Ocurrio una excepcion: " + e.getMessage());
10 }
11
```

Estructura try-catch-finally

El bloque `finally` garantiza la ejecución de su contenido sin importar si la operación dentro del `try` tuvo éxito o lanzó una excepción.

```
1  public class EjemploTryCatchFinally {
2      public static void main(String[] args) {
3          try {
4              System.out.println("1. Ejecutando codigo dentro del try
       ...");
5              int division = 10 / 0; // Lanza ArithmeticException
6          } catch (ArithmeticException e) {
7              System.err.println("2. Excepcion atrapada en el catch."
       );
8          } finally {
9              // Este bloque SIEMPRE se ejecuta
10             System.out.println("3. Bloque finally ejecutado con
       exito.");
11         }
12     }
13 }
14
```

Excepción: NullPointerException

Se produce al intentar acceder a un método o atributo a través de una referencia que apunta a `null`.

```
1  public class EjemploNullPointer {
2      public static void main(String[] args) {
3          String nombreUsuario = null;
4
5          try {
6              System.out.println("Longitud: " + nombreUsuario.length
       ());
7          } catch (NullPointerException e) {
8              System.err.println("Error: Se intento acceder a un
       objeto nulo.");
9          }
10     }
11 }
12
```

Excepción: ArithmeticException

Ocurre cuando se realiza una operación aritmética no válida en tiempo de ejecución, como una división entera entre cero.

```
1  public class EjemploArithmetic {
2      public static int calcularPromedio(int puntos, int alumnos) {
3          return puntos / alumnos;
4      }
5
6      public static void main(String[] args) {
7          try {
8              int promedio = calcularPromedio(150, 0);
9          } catch (ArithmeticException e) {
10             System.err.println("Error: No es posible dividir entre
       cero.");
11         }
12     }
13 }
14
```

Excepción: ArrayIndexOutOfBoundsException

Ocurre al intentar acceder a un índice fuera del rango válido de un arreglo (0 a longitud − 1).

```
1  public class EjemploArrayIndex {
2      public static void main(String[] args) {
3          int[] calificaciones = {85, 90, 78, 92}; // Indices validos
       : 0 a 3
4
5          try {
6              // Intento de acceso al indice 4 (fuera de limite)
7              System.out.println(calificaciones[4]);
8          } catch (ArrayIndexOutOfBoundsException e) {
9              System.err.println("Error: Indice fuera del limite del
       arreglo.");
10         }
11     }
12 }
13
```

Excepción: NumberFormatException

Se lanza cuando se intenta convertir una cadena de texto a un tipo numérico, pero la cadena no tiene el formato adecuado.

```
1  public class EjemploNumberFormat {
2      public static void main(String[] args) {
3          String entradaEdad = "25a";
4
5          try {
6              int edad = Integer.parseInt(entradaEdad);
7          } catch (NumberFormatException e) {
8              System.err.println("Error: '" + entradaEdad + "' no es
       un entero valido.");
9          }
10     }
11 }
12
```

Excepción: FileNotFoundException (Checked)

Excepción verificada que exige manejo obligatorio al intentar trabajar con archivos que podrían no existir en la ruta especificada.

```
1  import java.io.File;
2  import java.io.FileNotFoundException;
3  import java.util.Scanner;
4
5  public class EjemploFileNotFound {
6      public static void main(String[] args) {
7          try {
8              File archivo = new File("configuracion.txt");
9              Scanner lector = new Scanner(archivo);
10         } catch (FileNotFoundException e) {
11             System.err.println("Error E/S: El archivo no fue
       encontrado.");
12         }
13     }
14 }
15
```

Excepción: IllegalArgumentException

Utilizada en programación defensiva para validar argumentos recibidos en métodos y prevenir estados inconsistentes.

```
1  public class Alumno {
2      private double calificacion;
3
4      public void setCalificacion(double calificacion) {
5          if (calificacion < 0.0 || calificacion > 10.0) {
6              throw new IllegalArgumentException("Calificacion fuera
       de rango");
7          }
8          this.calificacion = calificacion;
9      }
10 }
11 // En el main:
12 // try { alumno.setCalificacion(15.5); }
13 // catch (IllegalArgumentException e) { System.err.println(e.
       getMessage()); }
14
```

Liberación de Recursos con finally (Lectura de Archivos)

El uso principal de `finally` en programación defensiva es asegurar el cierre de flujos de entrada/salida (archivos, conexiones) para evitar fugas de memoria.

```
1  import java.io.File;
2  import java.io.FileNotFoundException;
3  import java.util.Scanner;
4
5  public class LecturaSeguraArchivo {
6      public static void leerArchivo(String ruta) {
7          Scanner entrada = null;
8          try {
9              File archivo = new File(ruta);
10             entrada = new Scanner(archivo); // Lectura del archivo
11             while (entrada.hasNextLine()) {
12                 System.out.println(entrada.nextLine());
13             }
14         } catch (FileNotFoundException e) {
15             System.err.println("Error: Archivo no encontrado en " +
       ruta);
16         } finally {
17             if (entrada != null) {
18                 entrada.close(); // Liberacion garantizada del
       recurso
```

Garantía de Estado con finally (Escritura de Archivos)

Permite cerrar componentes de escritura como `PrintWriter` garantizando que los datos en el búfer se escriban en disco correctamente.

```
1  import java.io.FileNotFoundException;
2  import java.io.PrintWriter;
3
4  public class EscrituraSeguraArchivo {
5      public static void guardarLog(String mensaje) {
6          PrintWriter salida = null;
7          try {
8              salida = new PrintWriter("sistema.log");
9              salida.println("LOG: " + mensaje);
10         } catch (FileNotFoundException e) {
11             System.err.println("Error al crear el archivo de log.")
       ;
12         } finally {
13             if (salida != null) {
14                 salida.close(); // Asegura el volcado y cierre del
       archivo
15             }
16         }
17     }
18 }
19
```